Gym Management System

Database Management Systems Project

Eda Bodo Ghislaine
Matrikelnummer: 225786

Sommersemester 2026

Contents

1 Introduction

The Gym Management System is a database application developed for the Database Management Systems course.

The purpose of the application is to manage important information related to a gym.
The system supports the management of:

- members,
- courses,
- registrations,
- payments.

The application consists of a PostgreSQL database, a FastAPI backend and a web-based frontend.

2 System Architecture

The application follows a three-layer architecture:

1. Frontend
2. Backend
3. Database

The frontend communicates with the backend using HTTP requests.
The backend processes the requests and accesses the PostgreSQL database.
The general communication flow is:

$$\text{Frontend} \rightarrow \text{FastAPI Backend} \rightarrow \text{PostgreSQL}$$

The backend and the PostgreSQL database are executed using Docker Compose.

3 Technologies Used

The following technologies are used in the project:

- PostgreSQL
- FastAPI
- Python
- SQLAlchemy
- Docker
- Docker Compose

- HTML
- CSS
- JavaScript
- Git
- GitHub
- Debian packaging

4 Database Design

The database contains the following main entities:

4.1 Members

A member represents a person registered in the gym.
The member table contains:

- ID
- name
- email

4.2 Courses

A course represents a training course offered by the gym.
The course table contains:

- ID
- name
- trainer
- schedule

4.3 Registrations

A registration connects a member with a course.
The registration contains:

- registration ID
- member ID
- course ID

4.4 Payments

A payment stores payment information for a member.

The payment contains:

- payment ID
- member ID
- amount
- payment date

5 Backend

The backend is implemented with FastAPI.

The API provides CRUD operations for the main entities.

CRUD means:

- Create
- Read
- Update
- Delete

The API contains routes for:

- `/members/`
- `/courses/`
- `/registrations/`
- `/payments/`

6 API Security

Write operations are protected using an API key.

The HTTP header used by the application is:

```
X-API-Key
```

The operations POST, PUT and DELETE require the correct API key.

GET operations can be executed without an API key.

If the API key is missing or invalid, the server returns:

```
401 Unauthorized
```

7 Frontend

The frontend is implemented using HTML, CSS and JavaScript.

It provides sections for:

- Members
- Courses
- Registrations
- Payments

The frontend communicates with the FastAPI backend using the JavaScript `fetch()` function.

The backend runs on:

```
http://localhost:8000
```

The frontend development server runs on:

```
http://localhost:8080
```

8 Docker Environment

Docker Compose is used to start the backend and PostgreSQL database.

The main services are:

- `backend`
- `db`

The backend communicates with PostgreSQL through Docker's internal network.

9 Debian Package

The frontend can be packaged as a Debian package.

The generated package is called:

```
gym-management-frontend.deb
```

After installation, the frontend files are stored in:

```
/usr/share/gym-management/
```

The installed files are:

- `index.html`
- `style.css`
- `app.js`

10 Testing

The project contains automated API tests.

The tests verify:

- GET members
- GET courses
- GET registrations
- GET payments
- POST without API key
- POST with the correct API key

The security test verifies that a POST request without an API key returns:

```
401 Unauthorized
```

A POST request with the correct API key returns:

```
200 OK
```

11 Version Control

Git is used for version control.

The project is stored in a GitHub repository.

The repository contains the following main directories:

- `backend/`
- `frontend/`
- `packaging/`
- `tests/`
- `docs/`

Sensitive configuration information is stored in the `.env` file.

The `.env` file is excluded from the Git repository using `.gitignore`.

12 Conclusion

The Gym Management System demonstrates the interaction between a relational database, a REST API and a web frontend.

The application allows the management of members, courses, registrations and payments.

Docker simplifies the execution of the backend and database, while the API key protects write operations.

The project also includes automated tests, Debian packaging and Git version control.